Participant Rulebook & Problem Statement
Organized by ML Bangladesh

Hackathon Support/FAQ: https://t.me/+LWNi3Iftff05ZWU1 (Female),
https://chat.whatsapp.com/F3imIGh0E2THeVgu96LkxR (Male)

Welcome, builders

The whole game in one line: **build one useful thing that solves a real problem, using an open-source model, with an interface anyone can use.**

This is a solo hackathon and it runs for 4-6 hours. Nobody expects a finished startup by the end of it. A small, working, honest solution beats a huge unfinished one every single time.

Judges want to see how you think, what you built, and why it matters. Read this once, keep it open on your phone, and go build.

How to register

Registration, check-in and submission all happen on Major League Hacking (MLH). You must be registered to take part.

1. **Register.** Open the link below, log in or create an MLH account, and complete the registration form: https://events.mlh.com/events/15216-hacktoberfest-hack-day-chittagong-x-ieee-computer-society-iiuc-sbc/register
2. **Check in.** Arrive at IIUC before 9:30 AM(Female), 10:30 AM(Male) and check in with the organizers at the venue.
3. **Submit.** Before the deadline, submit your project and GitHub repo link on the MLH event page: https://events.mlh.com/events/15216-hacktoberfest-hack-day-chittagong-x-ieee-computer-society-iiuc-sbc

Event page: https://www.mlh.com/events/hacktoberfest-hack-day-chittagong-x-ieee-computer-society-iiuc-sbc

The 60-second version

In a hurry? These seven lines are the rulebook. Everything below just explains them.

1. **Work alone.** This is an individual hackathon, no teams.
2. **You get 4-6 hours.** Exact start and end times are announced at kickoff.
3. **Solve a campus or study problem.** Make student life better: anything on campus or in your studies that has no good solution yet. See "Problem statement" below.
4. **Use an open-source model.** Gemma(preferable), Llama, Qwen, Mistral, Whisper, YOLO and friends. This is required, so no closed model APIs.
5. **Give it a real interface.** A normal person should be able to use it without touching code.
6. **Keep everything in a GitHub repo** and commit as you go.
7. **Submit before the deadline.** A presentation round may follow; you'll know at kickoff.

How the day works

Everything happens in one sitting, from kickoff to submission in 4–6 hours, inside the event window of 10:00 AM – 5:30 PM. Exact times are left out of this rulebook on purpose, so the kickoff announcement is always the final word.

Stage	What happens	Pro tip
Check-in	You arrive at IIUC and check in with the organizers	Come early and grab a seat near a power outlet
Kickoff	Organizers share the timeline, confirm how t submit, and say whether there will be presentations	Write the deadline somewhere you'll keep seeir
Plan	You pick your problem and sketch what you'll build	Cap this at 30 minutes. Don't overthink it
Build	You code, test and commit. Mentors walk arc to help	Get one feature working end to end before addi more
Submit	You submit your project and GitHub repo link the MLH event page	Submit 15 minutes early. Wi-Fi loves to fail at th last minute
Present (maybe)	A short demo in front of the judges, only if announced	Have a 2-minute story ready anyway
Results	Judges review and announce the winners	Cheer for everyone. You all built something toda

Problem statement

Make campus life and studying better for university students, using open-source AI.

Every student at IIUC runs into the same small frustrations every week. Notices are scattered across Facebook groups and notice boards. Nobody is sure when the next bus to the city leaves. Good notes and past questions live in someone's phone. Simple paperwork means visiting three offices. Many of these problems have no proper solution on campus today.

Your challenge: pick one real problem that students face on campus or in their studies, and build an AI-powered tool that solves it. It can be at IIUC or any university, about classes, campus services, transport, admin, student life or wellbeing. If it makes a student's day easier, it counts.

Where to look

- **Studying and classes.** Summarizing lectures and slides, turning notes into practice questions, finding past questions, explaining hard topics in Bangla or English, exam planning, help with Arabic and Islamic studies courses.
- **Campus information.** Notices, class and room changes, the academic calendar, rules and policies, finding which office handles what.
- **Transport.** University bus schedules and routes between the city and Kumira, delays, crowding, finding the right bus from your area.
- **Admin and paperwork.** Course registration, fees, forms, certificates, and the steps nobody explains to freshers.
- **Student life.** Lost and found, clubs and events, hall and cafeteria information, buying and selling used books, finding study partners.
- **Wellbeing and access.** Stress and study balance, support for students with disabilities, helping new students settle in.
- **Careers.** CV feedback, internship search, interview practice, skill roadmaps for each department.

Example ideas

These are only to get you thinking. Don't copy them; your own problem is always stronger.

- A Bangla and English chatbot that answers questions from the academic calendar, rules and notices.
- A tool that turns photos of handwritten notes or lecture slides into a summary and a practice quiz.
- A Telegram bot that tells students which university bus to catch from their area, and when.
- A lost-and-found board that matches photos of found items to lost-item reports.
- A fresher's guide assistant that walks new students through registration, fees and campus offices.

Good to know

- **Stay on theme.** The problem must be about campus life or studying. A great project on an unrelated topic will lose points on "The problem".
- **Small and specific wins.** "Find past questions for CSE 3rd-semester courses" beats "AI for education".
- **Use sample data.** Don't scrape or upload real students' personal information. Make up realistic sample data instead.
- **Know your user.** Judges will ask which students have this problem and how they deal with it today. Have an answer.

What you're building

You choose the problem. Your project only needs three things.

1. **A real problem.** Something students actually face on campus or in their studies, at IIUC or any university (see "Problem statement"). Small and specific wins. "Bangla voice assistant that tells students which university bus to catch from their area" is far stronger than "AI for transport".
2. **An open-source model at the heart of it.** This is required. Gemma, Llama, Qwen, Mistral, Phi, Whisper, YOLO, BanglaBERT, anything that fits your problem. Run it locally, on Google Colab, through Ollama, or from Hugging Face. Also Free APIs (OpenAI, Gemini API and similar) are allowed. Whichever open model you pick, say so clearly in your README.
3. **An interface anyone can use.** A web app, mobile app, desktop app, a Telegram or WhatsApp bot, or a Gradio or Streamlit page are all fine. A terminal script or a Jupyter notebook alone is not enough. Quick test: if your ammu can't use it without your help, it needs a better interface.

Prize categories

On top of the main judging, these MLH prize categories are open at this event.

Category	What it's for	Prize / resources
Best Open-Source AI Project	An original project that uses open-source or open-weight AI as an important part of how it works For example: build with an open-weight large or sma language model, create an agent skill, or build or ad an open-source model harness.	Exclusive Hacktoberfest 20 Winner DEV Badge & MLH Swag Bag Build with DigitalOcean
Best Use of Gemma 4	See how much you can build with Gemma 4, a lightweight, multimodal open-weights model. Ideas: assistant that understands both text and images, or focused AI tool for learning, creativity, productivity o your community.	Build with Gemma 4

Links & resources

Resource	Link
Registration	https://events.mlh.com/events/15216-hacktoberfest-hack-day-chittagong-x-ie omputer-society-iiuc-sbc/register
Event page	https://www.mlh.com/events/hacktoberfest-hack-day-chittagong-x-ieee-comp r-society-iiuc-sbc
Check-in & submission	https://events.mlh.com/events/15216-hacktoberfest-hack-day-chittagong-x-ie omputer-society-iiuc-sbc
Gemma 4 resources (hub and docs overview)	https://mlh.link/gemma
Gemma quickstart	https://mlh.link/gemma-quickstart
Gemma docs	https://mlh.link/gemma-docs
Gemma beginner guide	https://mlh.link/gemma-beginnerguide
DigitalOcean	http://mlh.link/digitalocean
MLH Code of Conduct	https://static.mlh.io/docs/mlh-code-of-conduct.pdf

Everyone at the event follows the MLH Code of Conduct. Please read it before you arrive.

Your GitHub repo

Your repo is your proof of work, and judges will open it. Keep it simple and keep it honest.

- **One new repo, made for this hackathon.** Keep it public unless organizers say otherwise at kickoff.
- **Commit regularly**, every 30-45 minutes is a good rhythm. One giant commit at the last minute makes it hard for judges to see your work.
- **Never commit secrets.** No API keys, passwords or `.env` files. Add a `.gitignore`.
- **Don't upload model weights.** Link to the model on Hugging Face or Ollama instead.
- **Write a README.** This is the first thing judges read, so make it do the talking for you.

Copy this into your `README.md` and fill in the blanks:

```
# Project Name
One line: what it does and who it helps.

## The problem
Who faces it, and how they deal with it today.

## The solution
What your app does, in 3–4 sentences.

## How it works
- Model(s) used: name + link
- Tech stack: frontend, backend, tools
- Datasets (if any): name + link

## How to run it
1. Clone the repo
```

```
2. Install requirements
3. Run the app

## Screenshots / demo
Add 2–3 screenshots or a short GIF/video link.

## What's not done yet
Be honest. Known bugs, missing features.

## Credits
Libraries, tutorials, code snippets and anything else you used.
```

Fair play

One simple idea covers almost everything: **build it yourself, during the event, and be open about what you used.**

Totally fine	Not okay
Using AI coding assistants (ChatGPT, Claude, Copilot, Gemini) to help you write code	Submitting code you can't explain. Judges may ask how an part works
Open-source libraries, frameworks, starter templates a public datasets, with credit	Submitting a project you built before the hackathon
Pre-trained open-source models, fine-tuned or used as they are	Having someone else build it for you. This is a solo event
Googling, reading docs, tutorials and Stack Overflow	Copying another participant's code
Asking mentors and organizers for help	Faking a demo, like hardcoded answers shown as live mod output
Planning your idea and setting up your laptop before kickoff	Anything harmful, hateful, or that misuses people's perso data

About prep: you can install Python, Node, Ollama and your tools, and download models before kickoff (please do). Your project's actual code gets written during the hackathon.

Breaking these rules can get a project disqualified. We'd really rather not, so if you're unsure about something, just ask an organizer first.

Submission checklist

Run through this list before you hit submit. It takes five minutes and saves a lot of heartbreak.

- ☐ My GitHub repo link opens (test it in an incognito window)
- ☐ My README explains the problem, the solution and how to run it
- ☐ The README has screenshots, or a short GIF or video
- ☐ My app runs. I tested it one last time
- ☐ My app runs only on open-source models, and they're named and linked along with any datasets
- ☐ No API keys or passwords are anywhere in the repo
- ☐ My last commit is before the deadline
- ☐ I submitted my project on the MLH event page (link in "Links & resources")

Judges look at your repo as it stands at the deadline. Commits after that won't count.

How judging works

Judges look at your project as a whole, across the areas below. No hidden tricks. Aim to be solid in all of them instead of amazing at one and missing the rest.

Area	What judges look for	The question in their head
The problem	A real, clearly defined problem that matters to someone	"Who actually has this problem?"
Authenticity	Your own idea and your own work, and a solution t does what it claims	"Did they really build this, and does it rea work?"
Feasibility	A solution that can work with the tech, time and co needs	"Could this run for real users?"
Real-world fit	Something that makes sense today: the devices, internet, language and habits of real users	"Would people actually use this right nov
Design	An interface that is easy and pleasant to use	"Could a first-time user figure it out alon
Codebase	A clean, organized repo with a clear README and a honest commit history	"Can I understand and run this?"
Presentation	A clear story and a working demo, if presentations happen	"Did they explain it well in a few minutes

If there's no presentation round, judges go by your repo and your app. That's one more reason to make your README great.